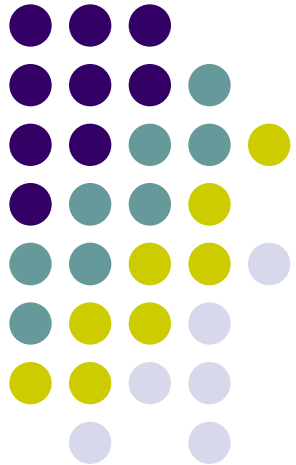
OPERATING SYSTEMS

F3d-308



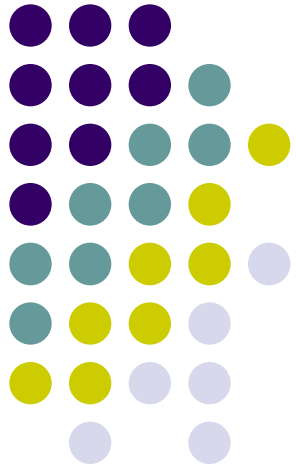
许言午

Frank, Yanwu XU

xuyanwu@scut.edu.cn

B1c-501

Chapter 1 Introduction



1.1 What is an operating system
1.2 History of operating systems
1.3 Computer hardware review
1.4 The operating system zoo
1.5 Operating system concepts
1.6 System calls
1.7 Operating system structure

What is a Computer?

- A computer is a digital electronic machine that can be programmed to carry out sequences of arithmetic or logical operations automatically.









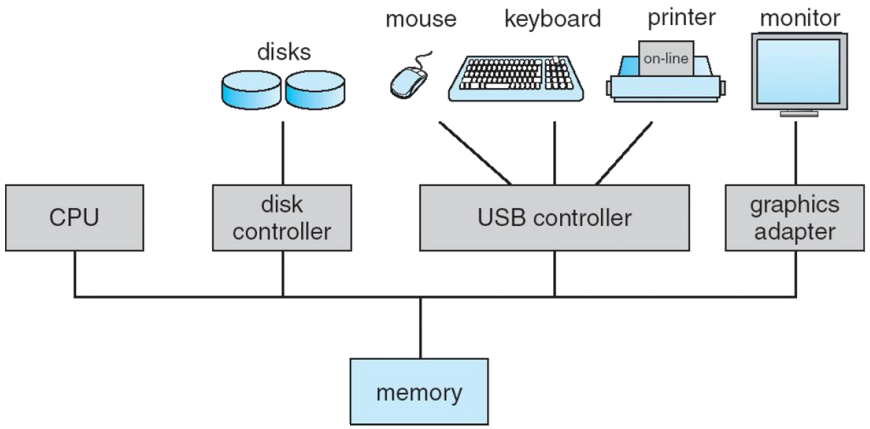


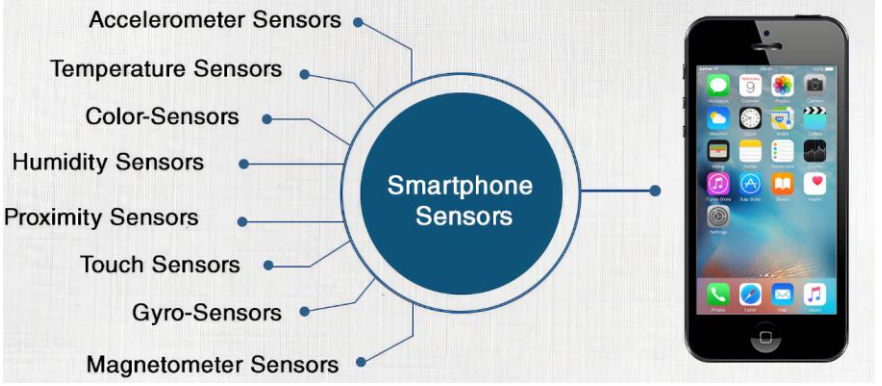




Computer System Organization

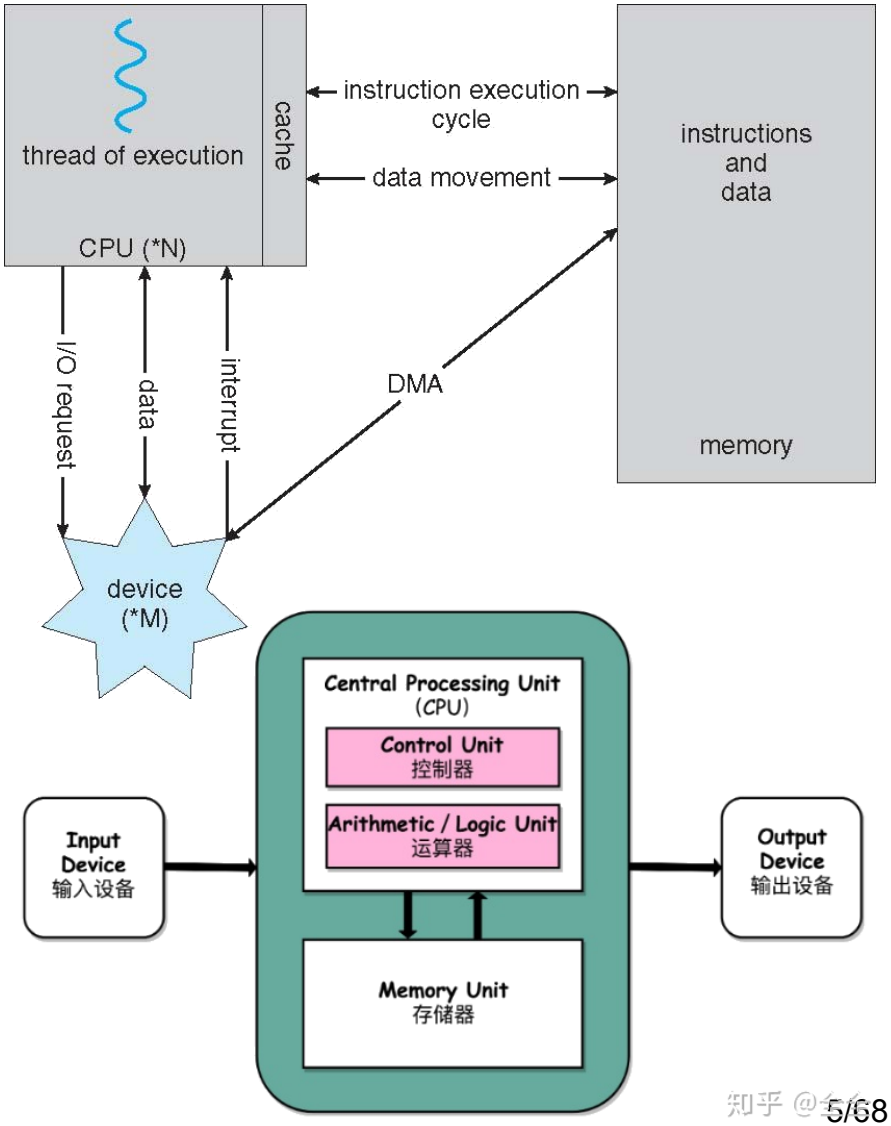
- One or more CPUs, device controllers connect through common bus providing access to shared memory
- Concurrent execution of CPUs and devices competing for memory cycles





How a Modern Computer Works

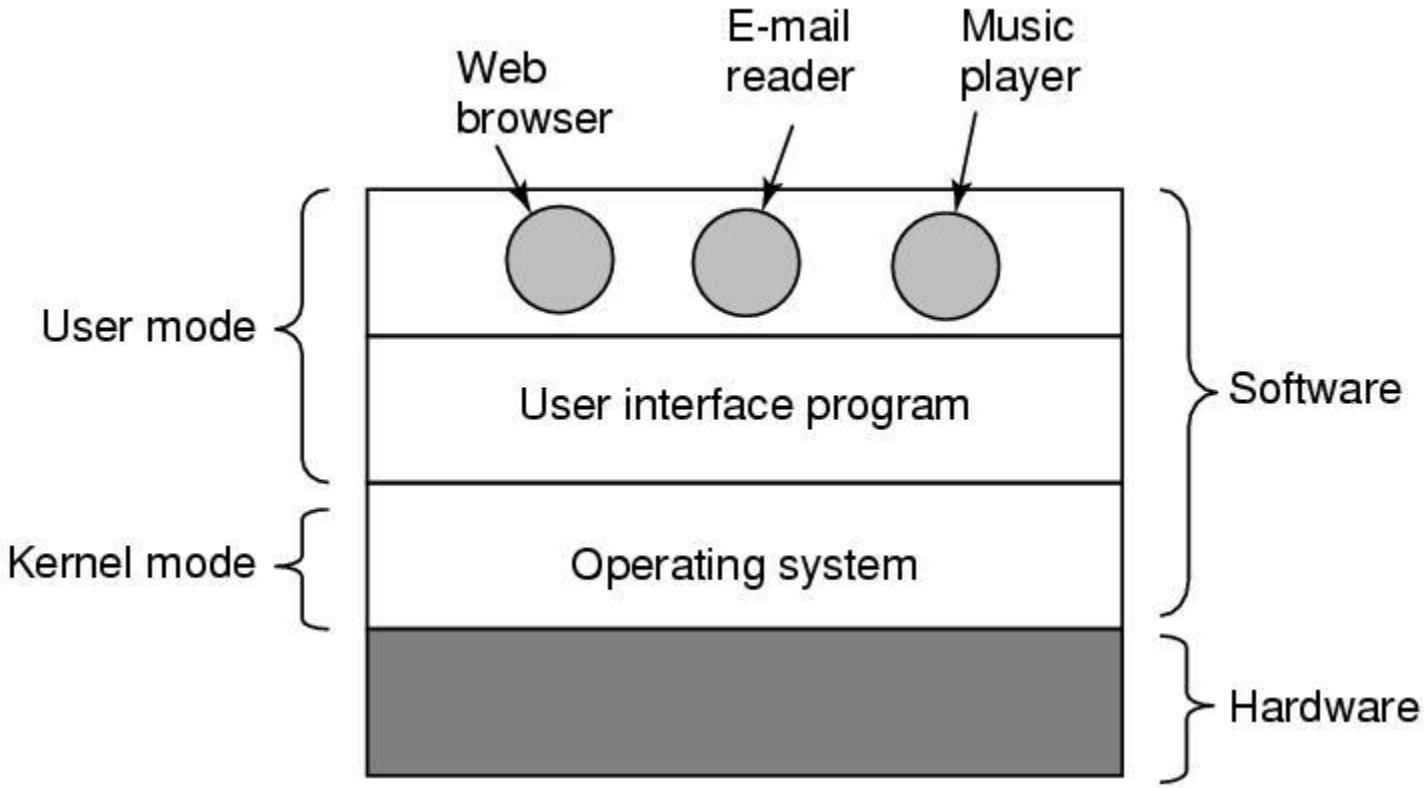
- **von Neumann Architecture:**
  - a single, shared memory for programs and data
  - a single bus for memory access
  - an arithmetic unit
  - and a program control unit



John von Neumann (1903~1957)

- Mathematician, computer scientist, physicist, chemist
- Known for invention of modern computer architecture and game theory

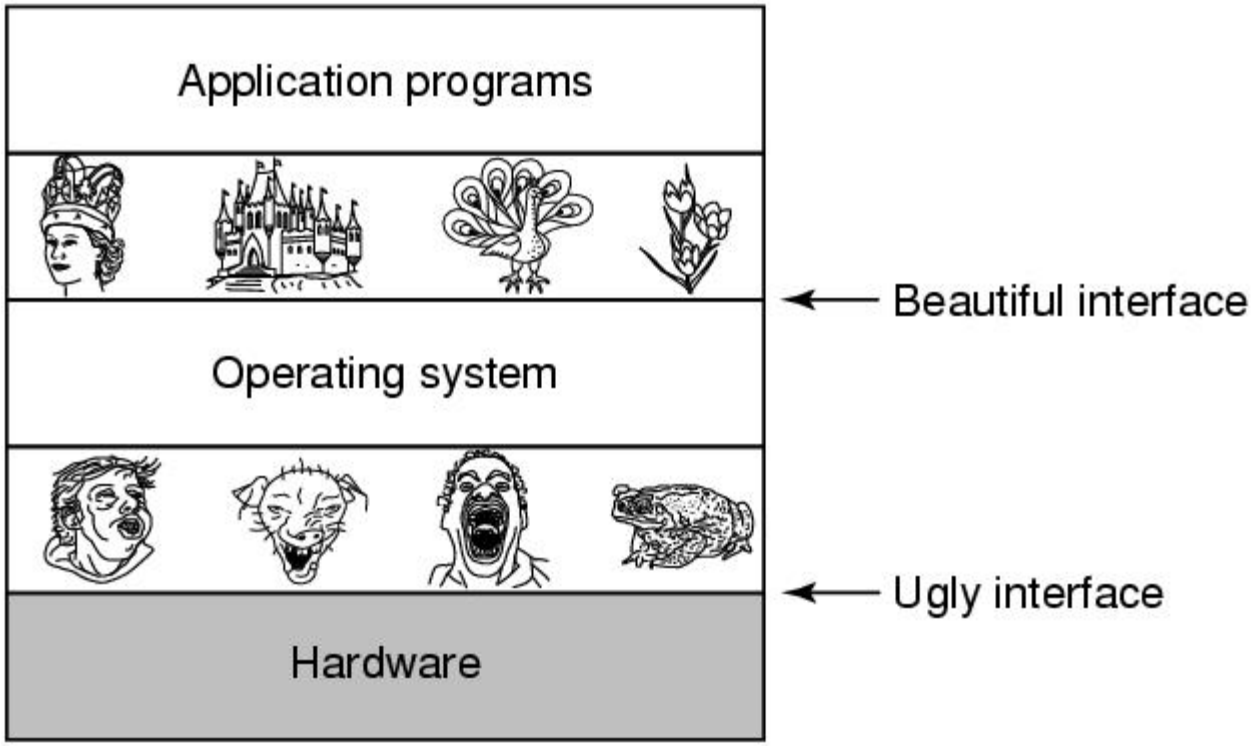
1.1 Where?



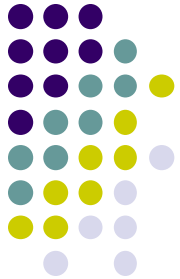
- A computer system consists of
  - Hardware
  - Software

What is OS? (1/2)

- It is an extended machine (top-down)
  - Hides the messy details which must be performed
  - Presents user with a virtual machine, easier to use

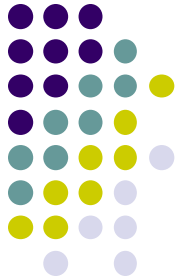


What is OS? (2/2)



- It is a resource manager (bottom-up)
  - Allow multiple programs to run at the same time
  - Manage and protect memory, I/O devices, and other resources
  - Includes multiplexing (sharing) resources in two different ways:
    - In time
    - In space

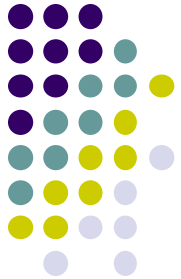
1.2 History of Operating Systems



- First generation 1945 - 1955
  - vacuum tubes, plug boards
- Second generation 1955 - 1965
  - transistors, batch systems
- Third generation 1965 – 1980
  - ICs (Integrated Circuits) and multiprogramming
- Fourth generation 1980 – present
  - personal computers and workstations (Network operating systems and distributed operating systems)
- Fifth generation 1990 – present
  - Mobile Computers

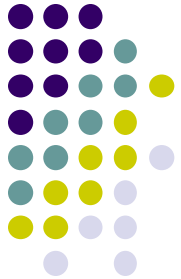
Requirement —> development

History of Operating Systems (1)

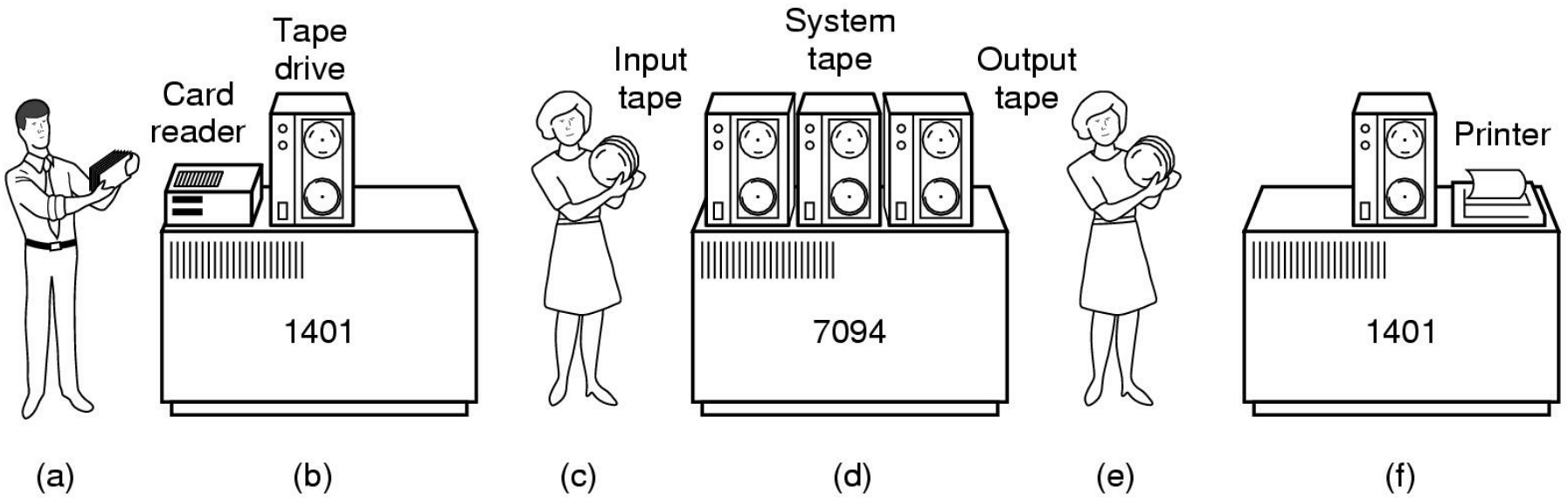


- **Vacuum tubes, handwork**
- **mode**
  - User: operator and programmer
  - Machine language
  - I/O: plugboard, punched cards
- **characteristic**
  - The user occupy the machine alone
  - CPU wait the user load and unload the plugboard
- **conflict**
  - CPU (high) ----- handwork (low)
- **Method**
  - Special operator, batch system

History of Operating Systems (2)



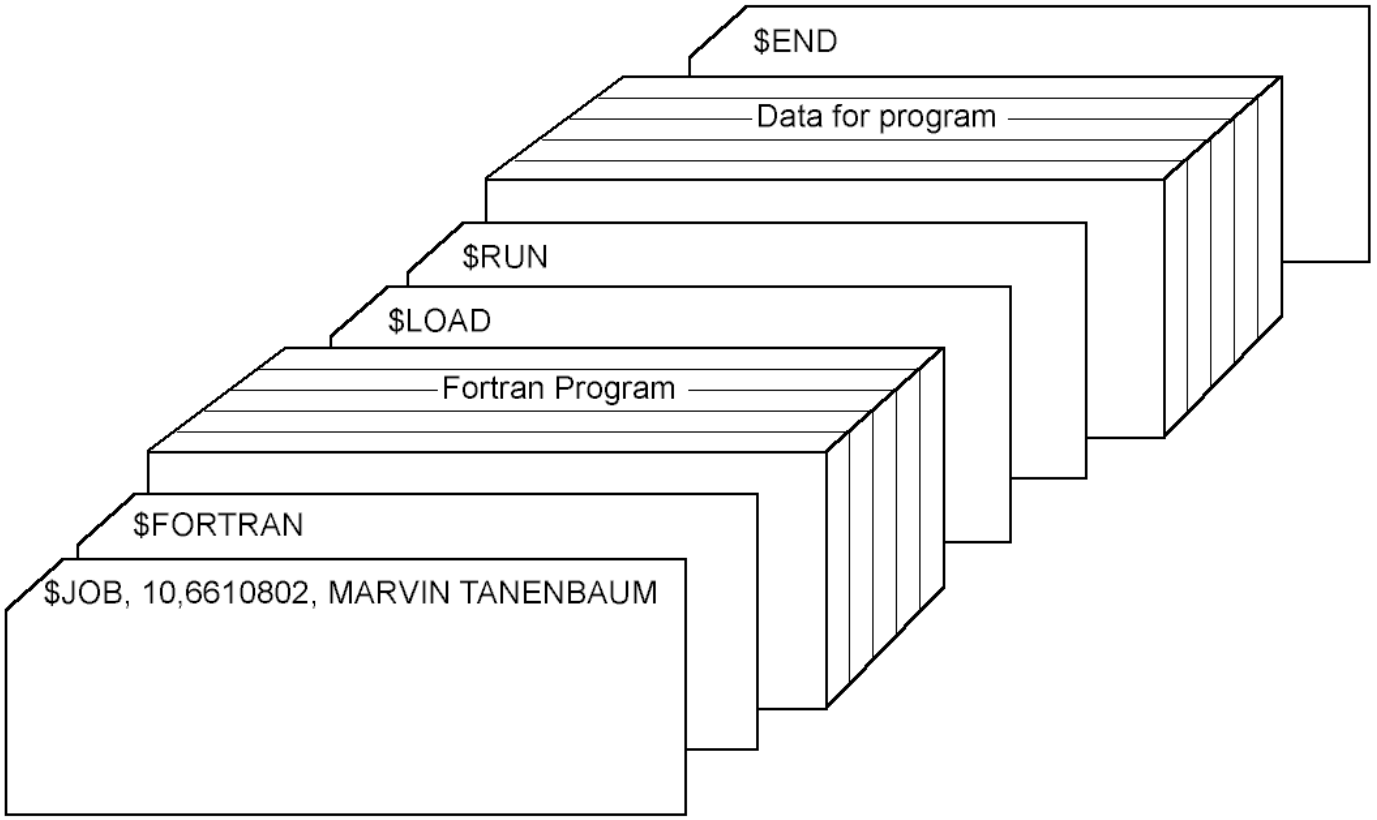
- Transistors, batch system
- Mode
  - User: operator or programmer
  - Language: assembly language, Fortran…
  - I/O: card, tape
  - os: Monitor (execute the jobs)
  - online batch system / off-line batch system



Early off-line batch system

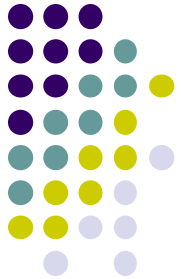
- (a) Programmers bring cards to 1401
- (b) 1401 reads batch of jobs onto tape
- (c) Operator carries input tape to 7094
- (d) 7094 does computing
- (e) Operator carries output tape to 1401
- (f) 1401 prints output

Job: program, data, job specification



- Structure of a typical FMS job – 2nd generation

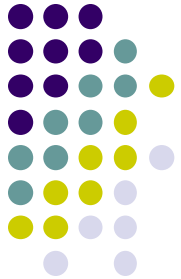
History of Operating Systems (3)



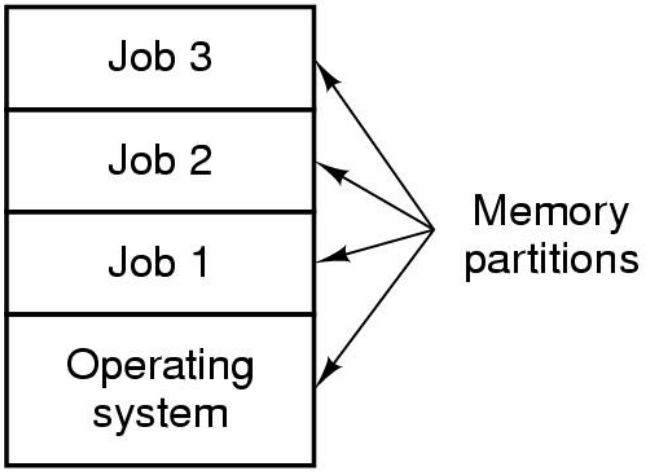
- IC, multiprogramming
- Batch system: good efficiency, bad interaction.
- time-sharing system

  (a variant of multiprogramming):
  - multi users share the computer, each has a online terminal;
  - Allocate CPU by time slice(时间片);
  - quick response time.

History of Operating Systems (3)

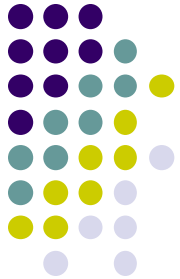


- Mode
  - Partition memory into several pieces, with a different job in each partition
  - Parallel in macroscopic
  - Serial in microcosmic



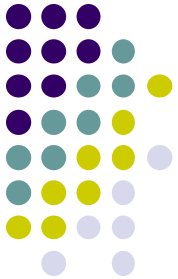
- Multiprogramming system
  - three jobs in memory – 3rd generation

Fourth generation 1980 – Present



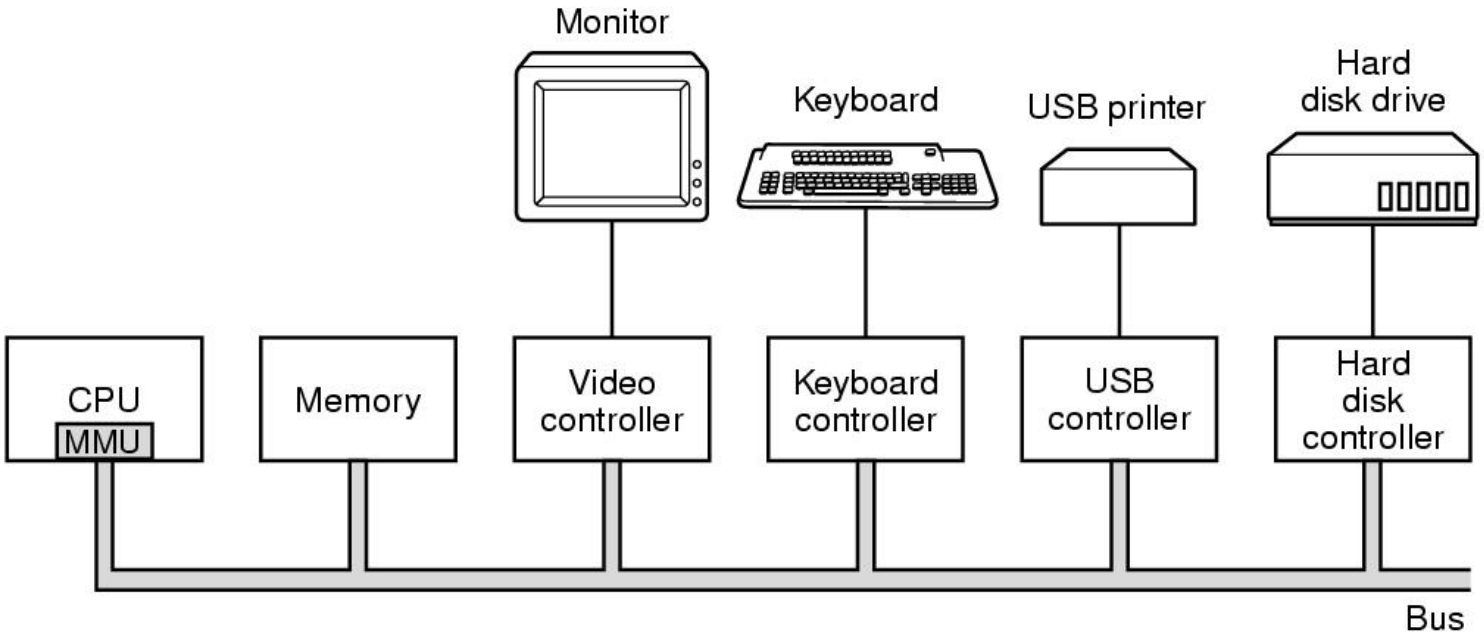
- Large scale integration (LSI)
- Operating systems for personal computers
  - CP/M (Control Program for Microcomputers)
  - DOS (Disk Operating System) ➔ MS-DOS
  - GUI (Graphical User Interface)
  - Windows ➔ Windows 95/98/Me
  - Windows NT➔ Windows 2000➔ Windows XP➔ Vista ➔Win7 ➔Win8 ➔Win10 ➔Win11
  - UNIX:
    - X Windows
    - Linux, GNOME, KDE

Fifth generation 1990 – Present



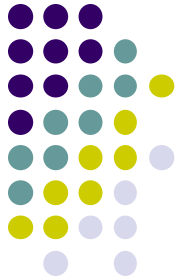
- Operating systems for mobile computers
  - PDA(Personal Digital Assistant)
  - Smart Phone
    - Symbian
    - Blackberry
    - iOS
    - Android
    - Windows Phone
    - Harmony

1.3 Computer Hardware Review



- Some of the components of a simple personal computer

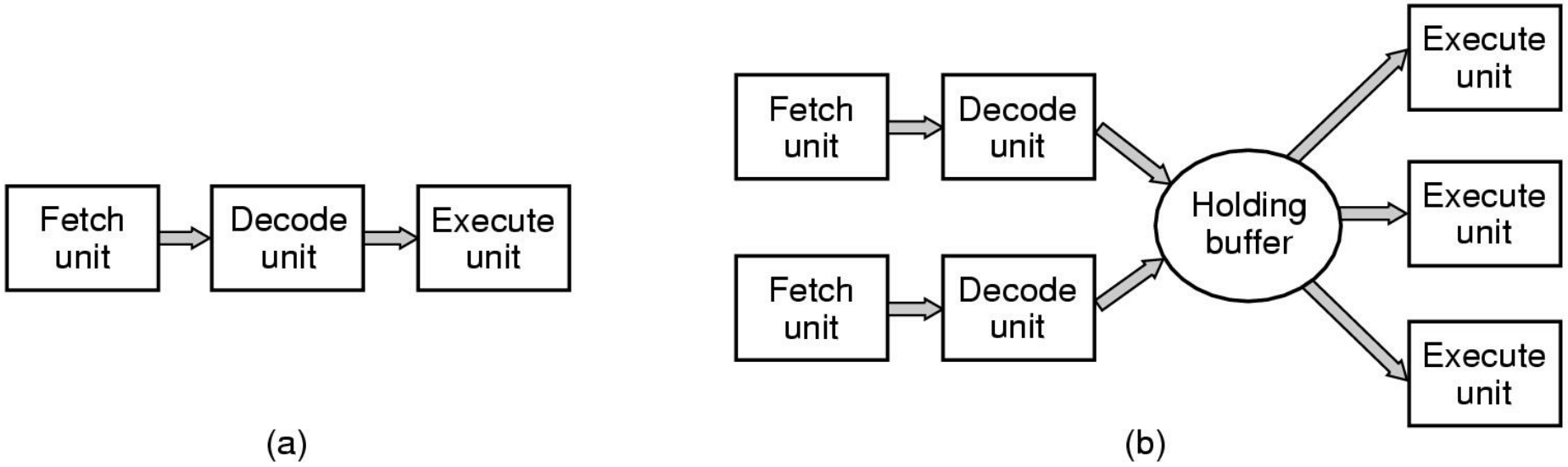
Computer Hardware Review (CPU)



calculator, controller, registers, cache.

- Registers: (quick, high price, low capability)
  - data register
  - address register
  - PC：Program Counter
  - IR：Instruction Register
  - PSW：Program Status Word

- Process of instructions



(a) A three-stage pipeline
(b) A superscalar (超标量) CPU

- Multithreaded and Multicore Chips

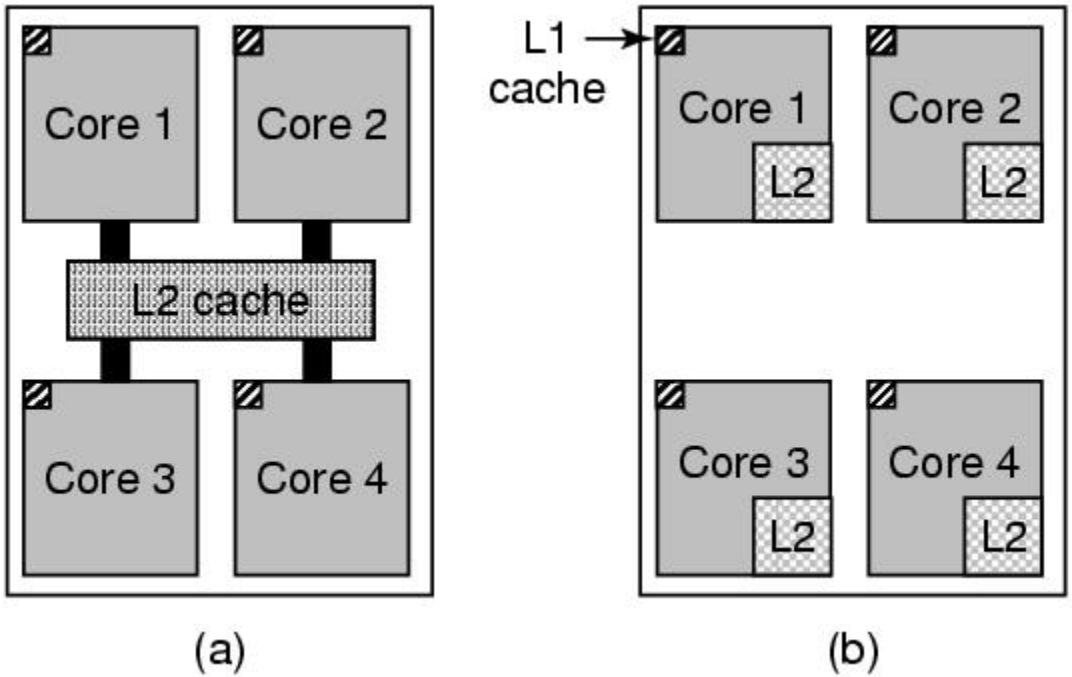
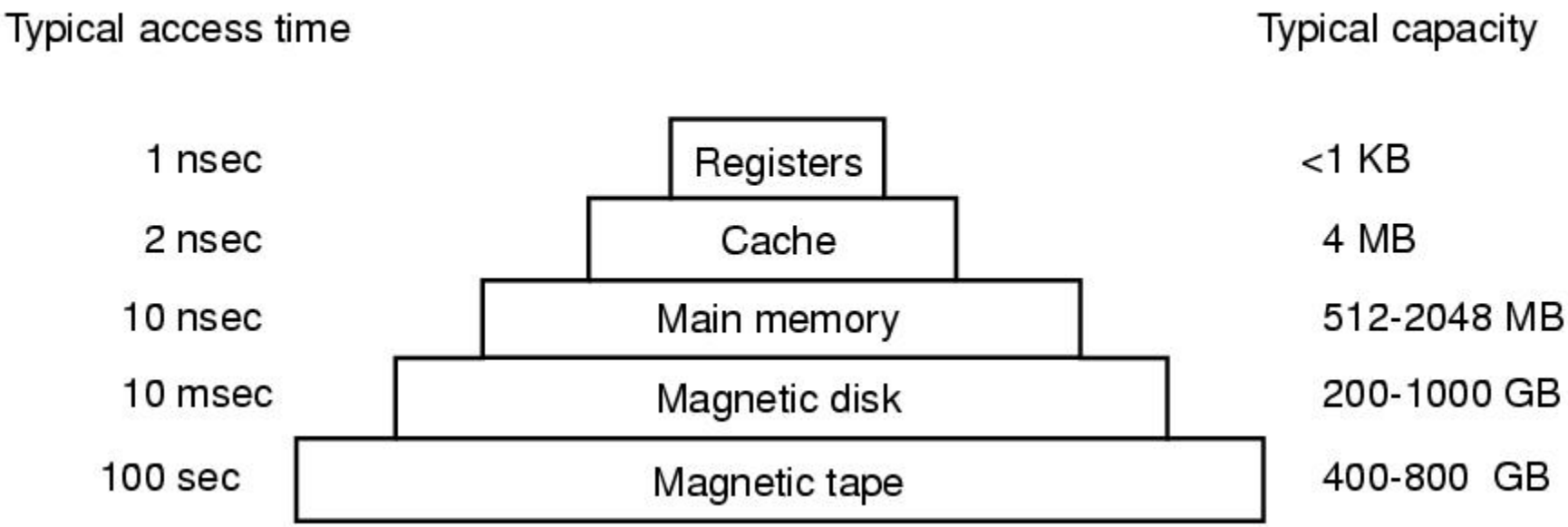


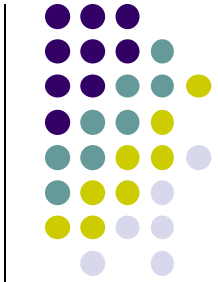
Figure 1-8. (a) A quad-core chip with a shared L2 cache. (b) A quad-core chip with separate L2 caches.

Computer Hardware Review (Memory)

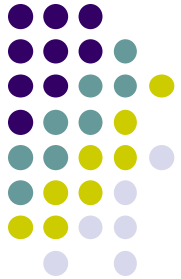
Capability, speed, cost



- A typical memory hierarchy
  - The numbers shown are rough approximations

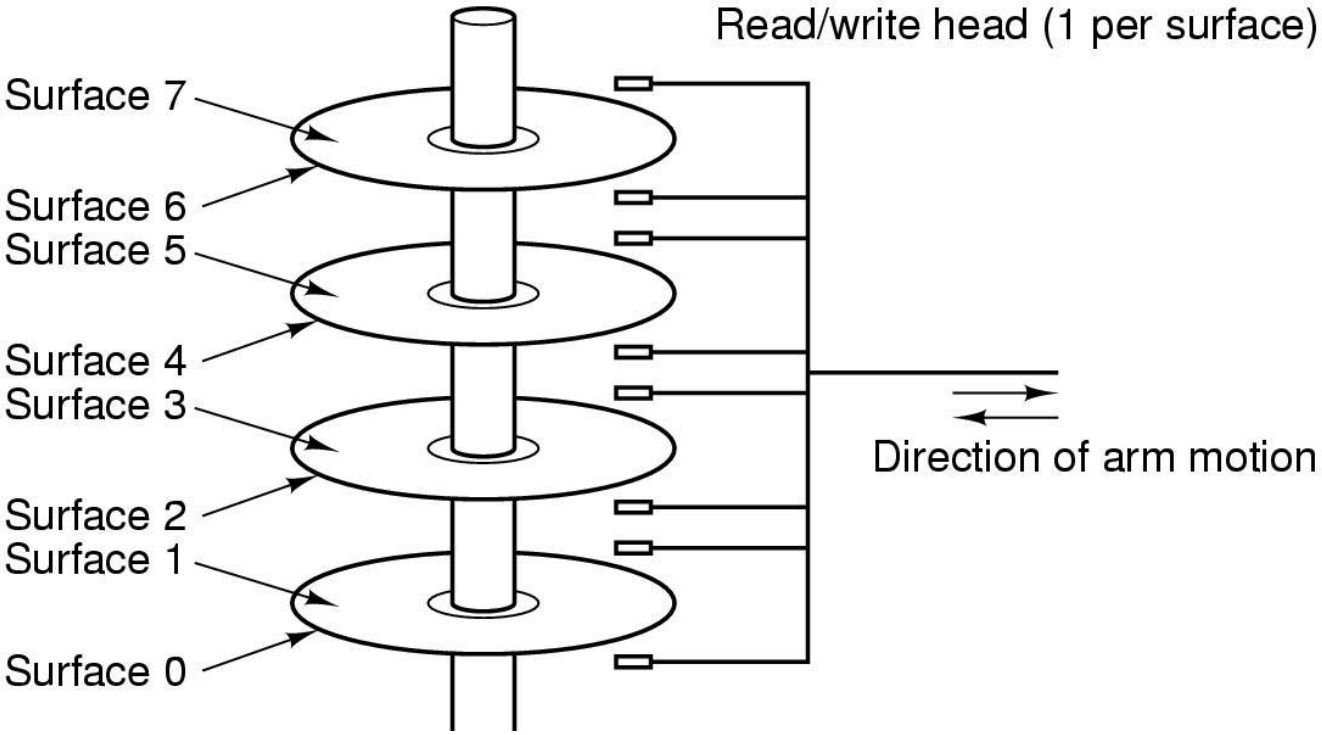


- Cache
  - Questions when dealing with cache:
    - When to put a new item into the cache.
    - Which cache line to put the new item in.
    - Which item to remove from the cache when a slot is needed.
    - Where to put a newly evicted item in the larger memory.
  - Multi-level cache (2-level or 3-level)



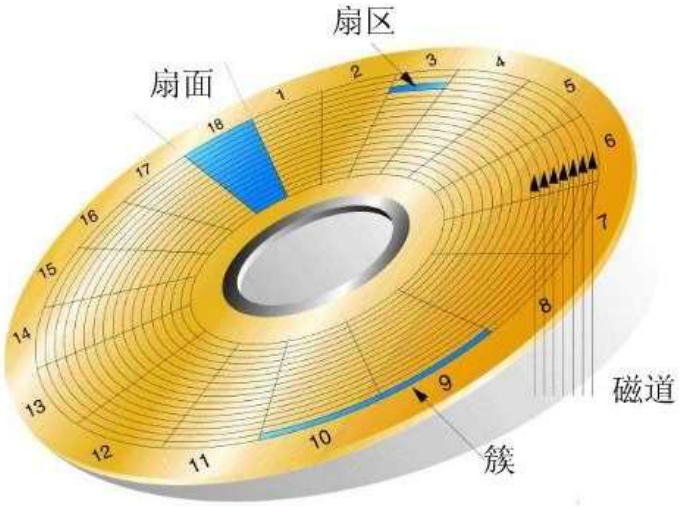
- RAM
  - With two or more programs in main memory at once, two problems must be solved:
    - How to protect the programs from one another and the kernel from them all
    - How to handle relocation

- Disk

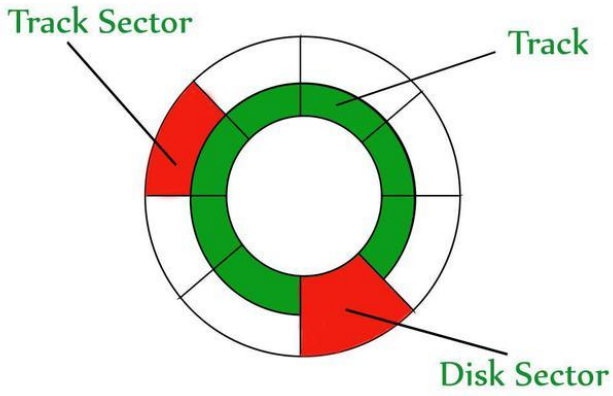


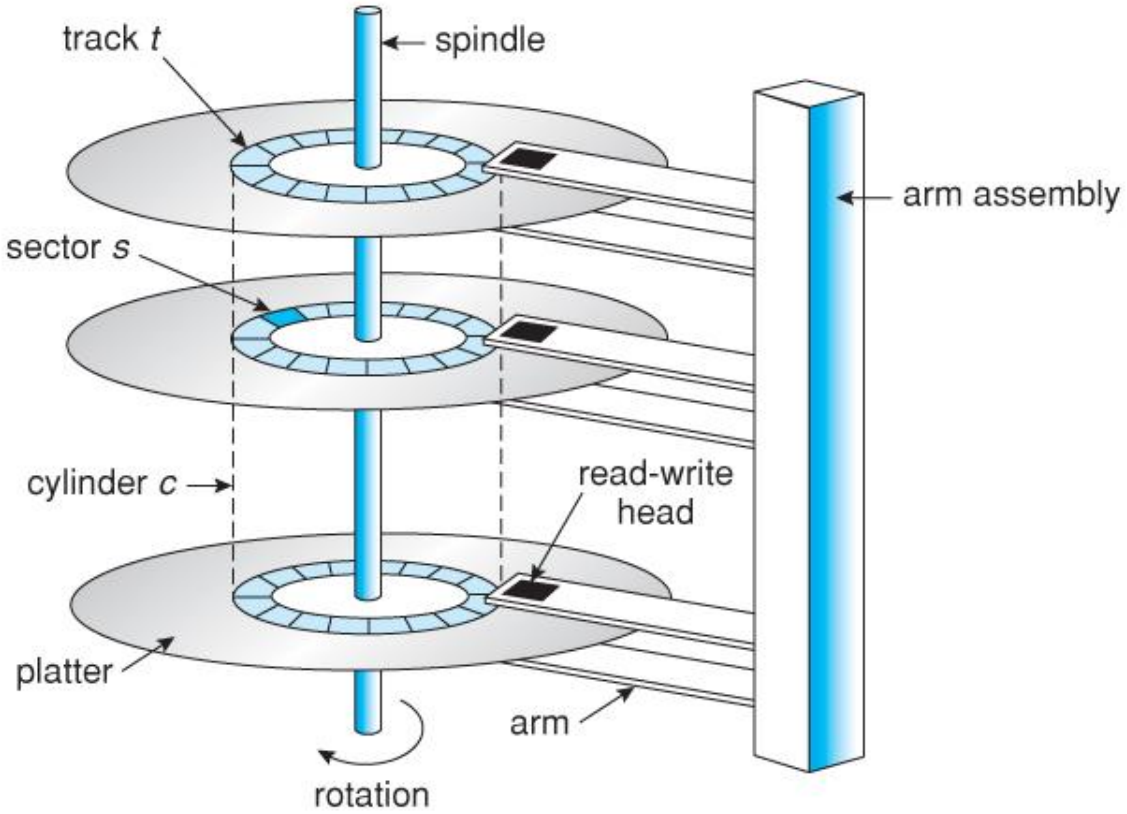
Structure of a disk drive

- Disk

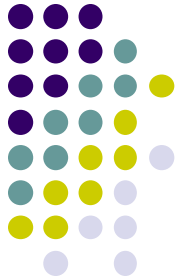


磁盘上的磁道、扇区和簇

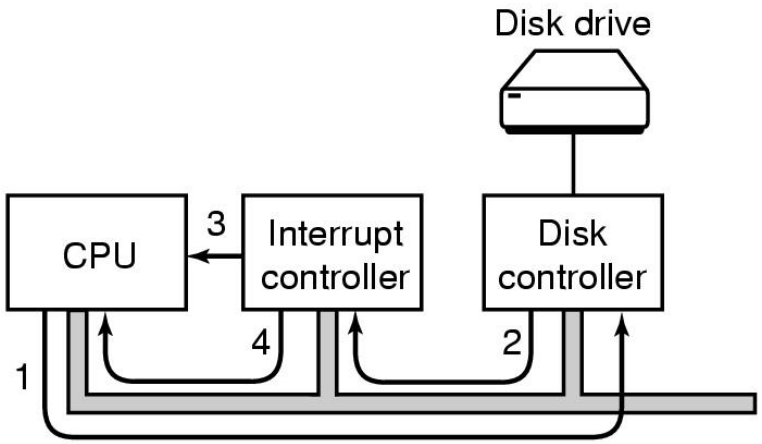




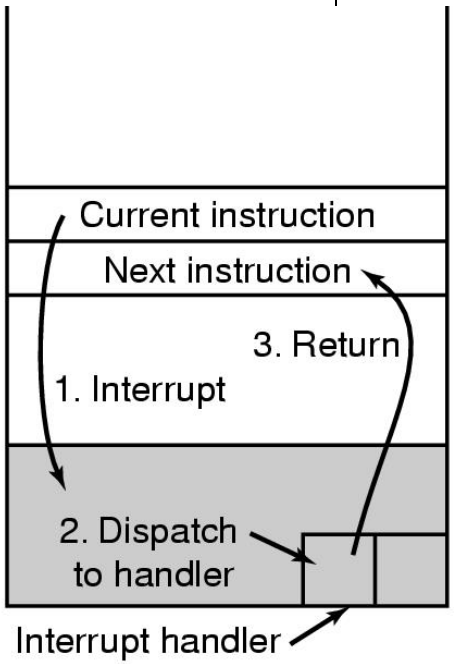
Computer Hardware Review (I/O)



- I/O devices generally consist of two parts: a controller and the device itself.
  - Device driver: the software that talks to a controller, giving it commands and accepting responses.
- Input and output can be done in three different ways:
  - Program-controlled I/O (Polling)
  - Interrupt-driven I/O
  - Direct Memory Access (DMA)



(a)

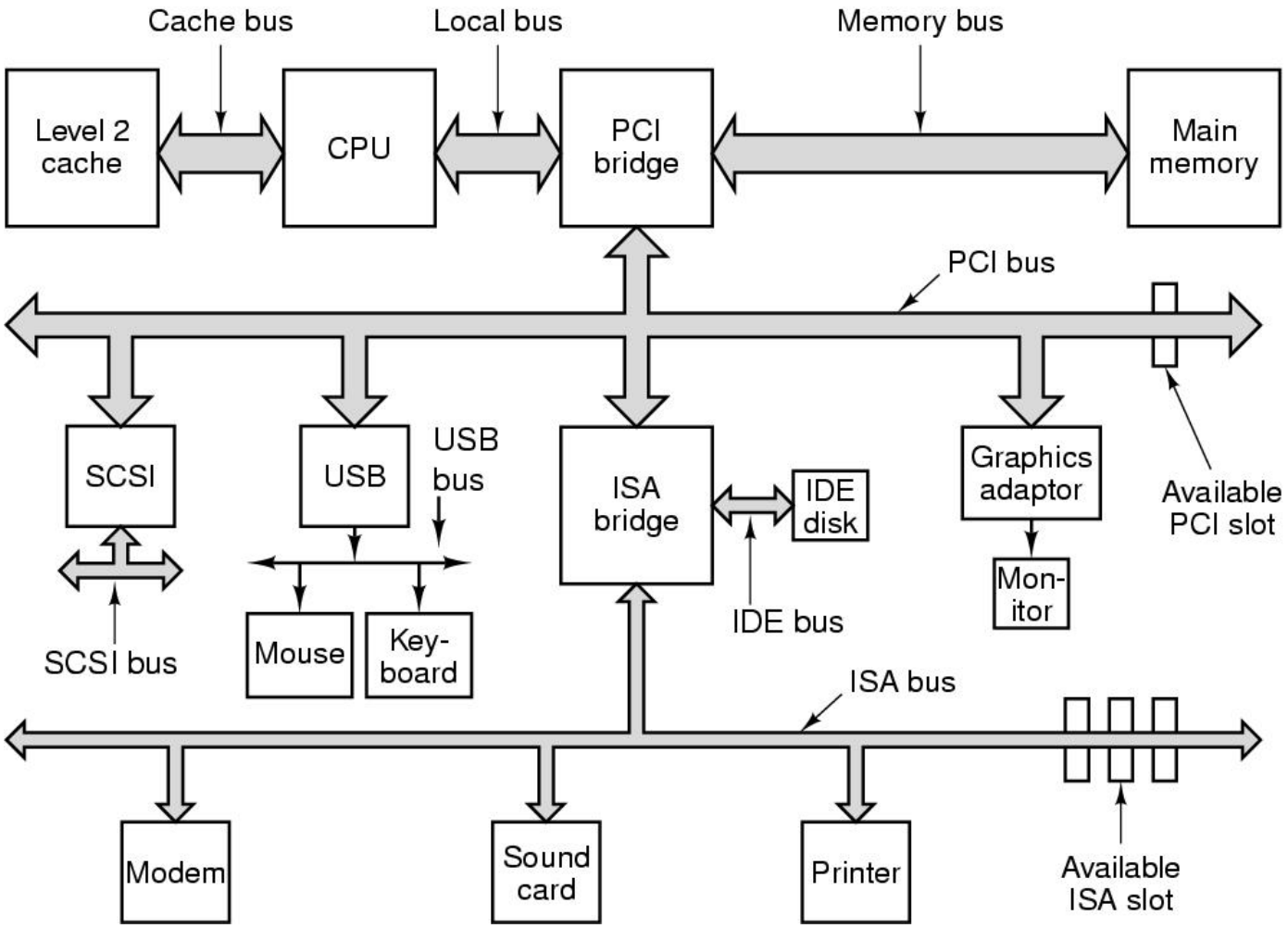


(b)

(a) The steps in starting an I/O device and getting interrupt

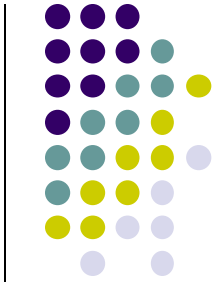
(b) How the CPU is interrupted

Computer Hardware Review (Buses)

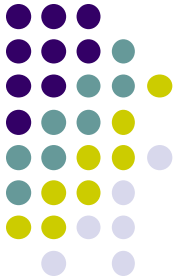


The structure of a large Pentium system

1.4 The Operating System Zoo

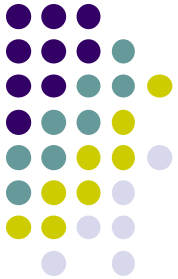


1. Mainframe operating systems
   - E.g.: OS/390, zOS
2. Server operating systems
   - E.g.: Solaris, FreeBSD, Linux, Windows Server 200x
3. Multiprocessor operating systems
4. Personal computer operating systems
   - E.g.: Win 7/8/10/11, Vista, Macintosh, Linux, FreeBSD
5. Handheld operating systems
   - E.g.: Android, iOS



6. Embedded operating systems
   - E.g.: QNX, VxWorks
7. Sensor node operating systems
   - E.g.: Tiny OS
8. Real-time operating systems
   - Hard real-time system: the action absolutely must occur at a certain moment, such as e-Cos.
9. Smart card operating systems
   - Java Virtual Machine (JVM)

1.5 Operating System Concepts (Processes)



- A process is a program in execution.
- 为了描述和控制进程的运行，系统为每个进程定义了一个数据结构——进程控制块PCB (Process Control Block)。所谓系统创建一个进程，就是由系统为某个程序设置一个PCB，用于对该进程进行控制和管理。进程任务完成，由系统收回其PCB，该进程便消亡。系统将根据某PCB而感知相应进程的存在，故说PCB是进程存在的唯一标志。
- The information about a process (PCB) is stored in an operating system table called the **process table**.

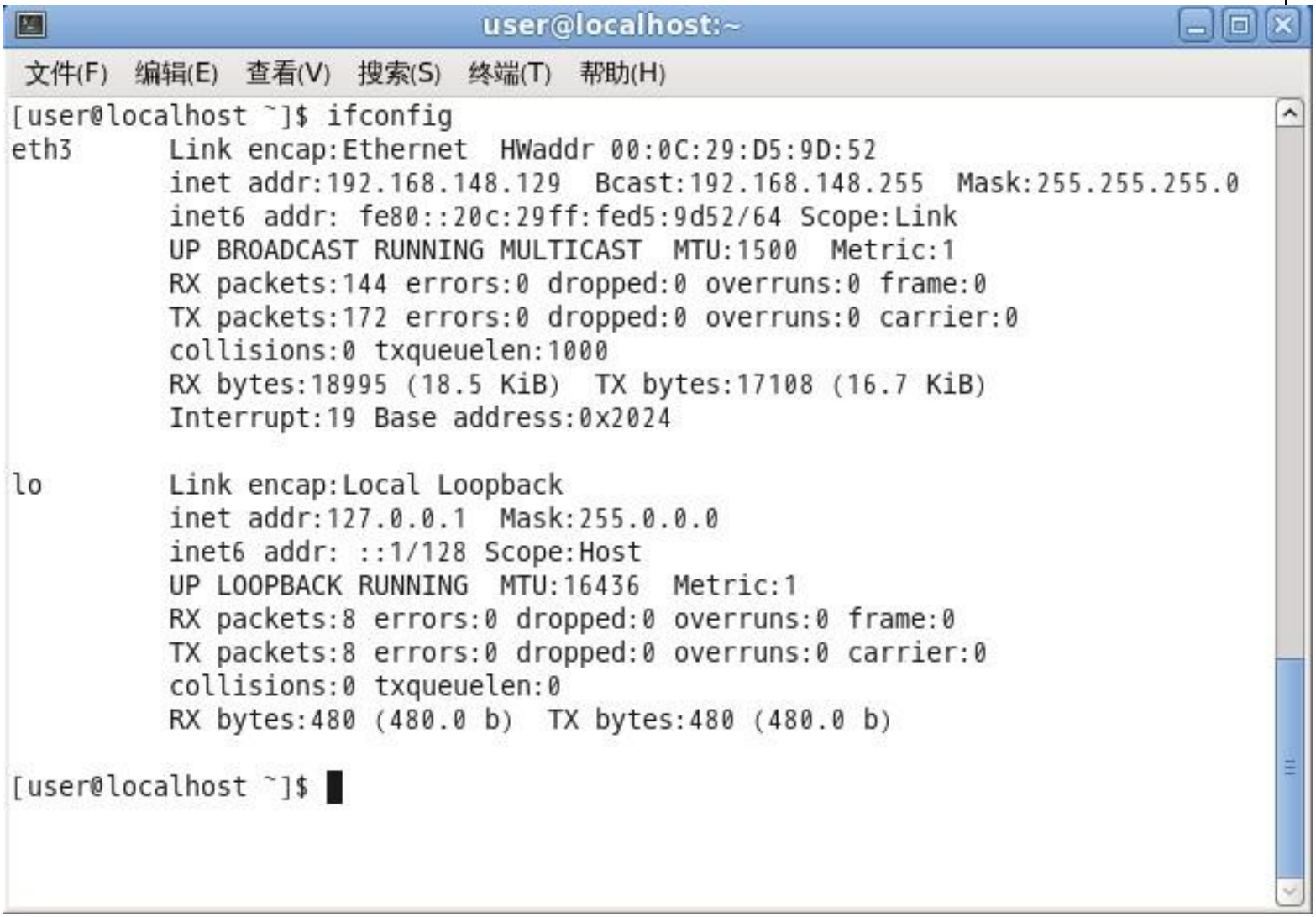
Operating System Concepts (Processes)



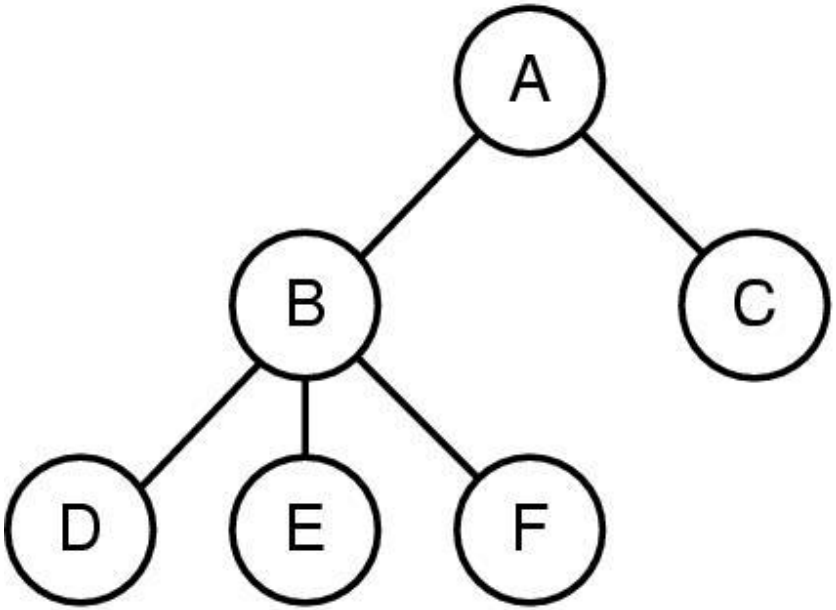
- A process called the **command interpreter** or **shell** read commands from a terminal.
- If a process can create one or more other process (child processes) and these process create child processes, we can build them in a tree structure.
- In UNIX, there is a parent process for all processes.

Operating System Concepts (Processes)



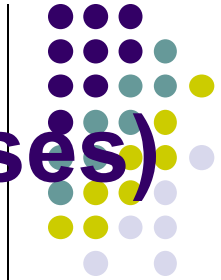


Operating System Concepts (Processes)



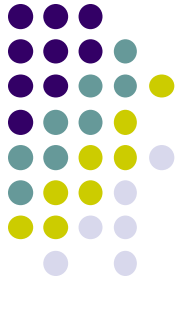
- A process tree
  - Process A created two child processes, B and C
  - Process B created three child processes, D, E, and F

Operating System Concepts (Processes)



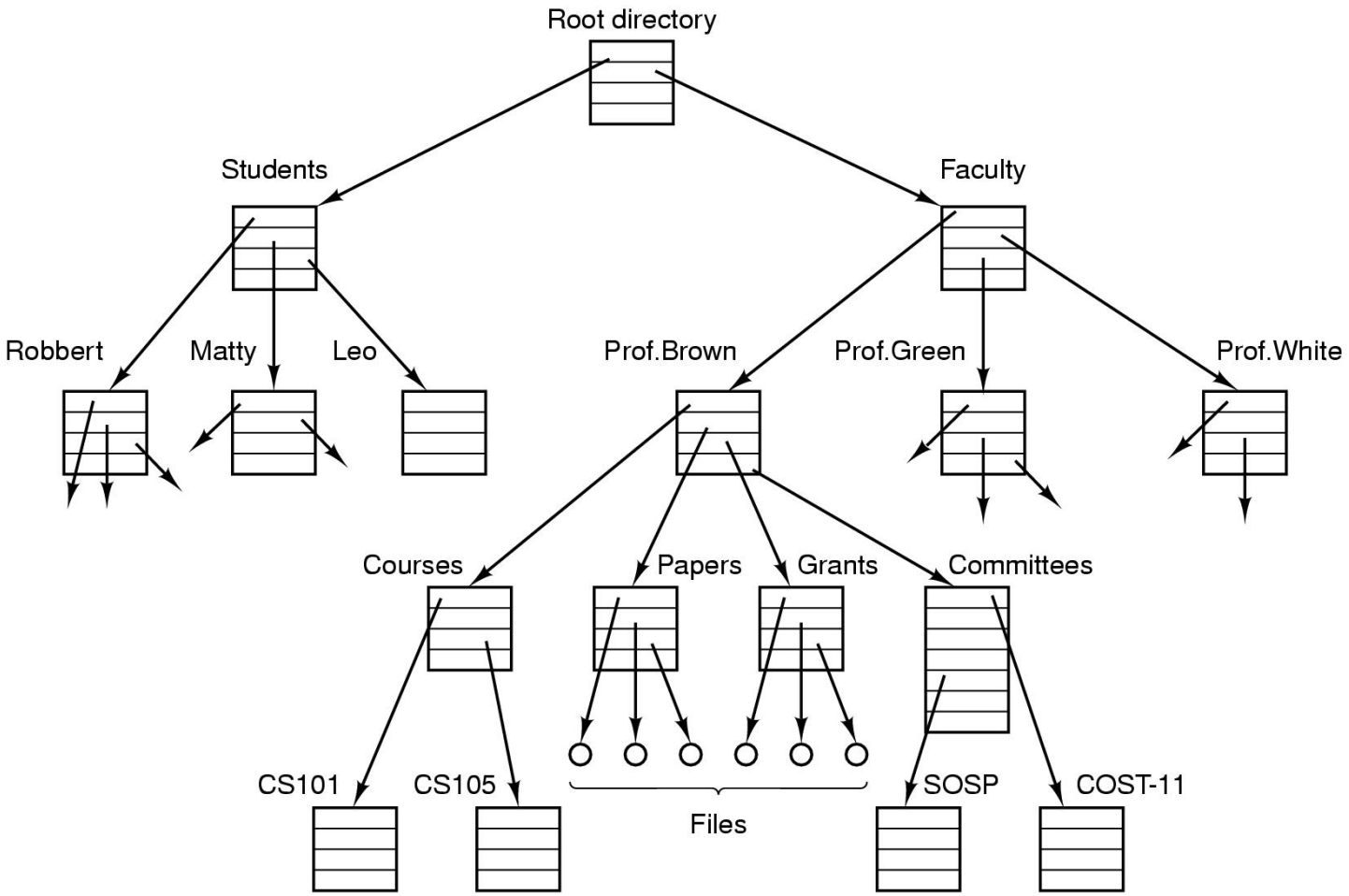
- Process have many status: running, blocked, ready.
- Each person authorized to use a system is assigned a **UID** (User IDentification). A process is assigned a **PID** (Process IDentification).
- Users can be members of groups, each of which has a **GID** (Group IDentification).
- The **superuser** (root in UNIX) has special privilege to violate many of the protection rules.
- The communication to cooperate and synchronize processes is called **interprocess communication**.

Operating System Concepts (Address Spaces & Files)



- **Address Space**: a list of memory allocations from 0 to some maximum, which the process can read and write.
- In **virtual memory** the operating system keeps part of the address space in main memory and part on disk.
- A **file system** provides users with a nice, clean abstract model of device-independent files.
- A directory is used to group files together.
- A path name can specify the location of a file.
- A root directory is the top of the directory hierarchy. In Unix, / is the root directory.
- Each process has a current **working directory** (pwd).

Operating System Concepts (Files)



A file system for a university department

Operating System Concepts (Files)

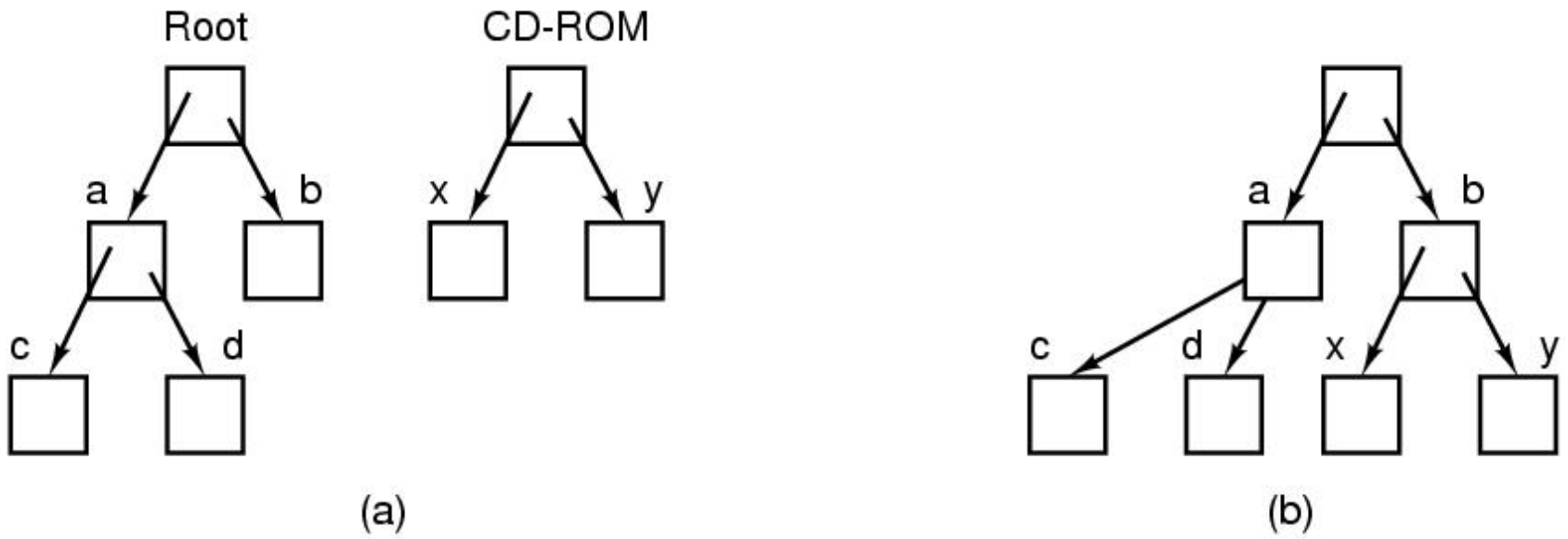


- If a file access is permitted, the system return small integer called a **file descriptor** to use in subsequent operations.

- Two kinds of special files exist in UNIX under the /dev directory: **block special files** (disks) and **character special files** (printers, modems).

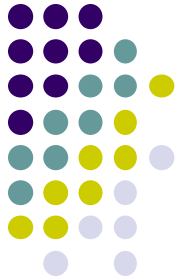
- In UNIX the mounted file system is used to deal with removable media (mount).

Operating System Concepts (Files)

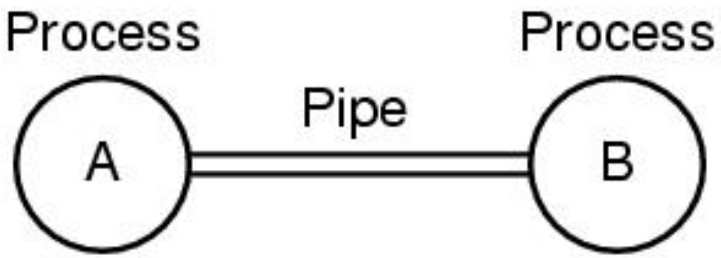


- Before mounting,
  - The files on the CD-ROM are not accessible
- After mounting,
  - The files on the CD-ROM are part of the file hierarchy

Operating System Concepts (Files)

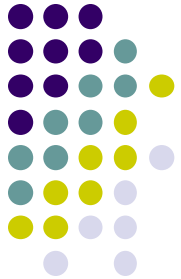


- A pipe is a sort of pseudofile that can be used to connect two processes.



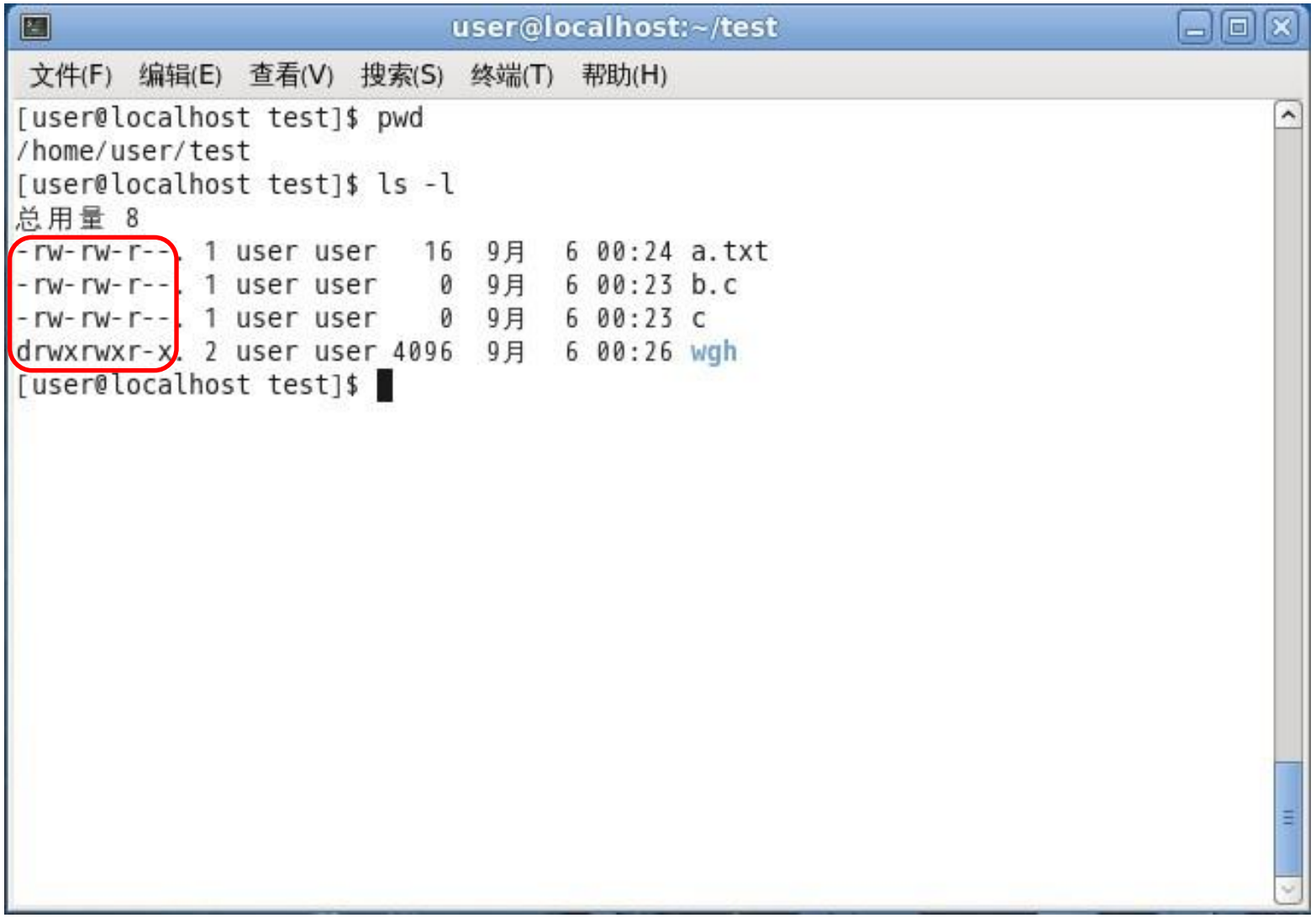
Two processes connected by a pipe

Operating System Concepts (Protection & Shell)

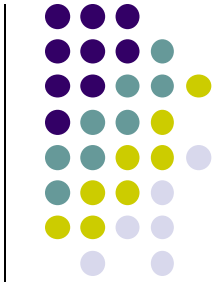


- Files in UNIX are protected by a 9-bit binary protection code.
  - The protection code consists of three 3-bit fields, one for owner, one for group, and one for others.
  - Each field has a bit for read, a bit for write, and a bit for execute. They are known as rwx bits.
- The UNIX command interpreter is called the **shell**.
- Many shells exist, including sh, csh, ksh, and bash.
- The user can use < and > to redirect the standard input and output. The ampersand (&) can be put after a command to start it as a background job.

Operating System Concepts (Protection & Shell)

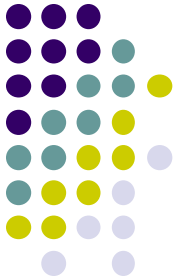


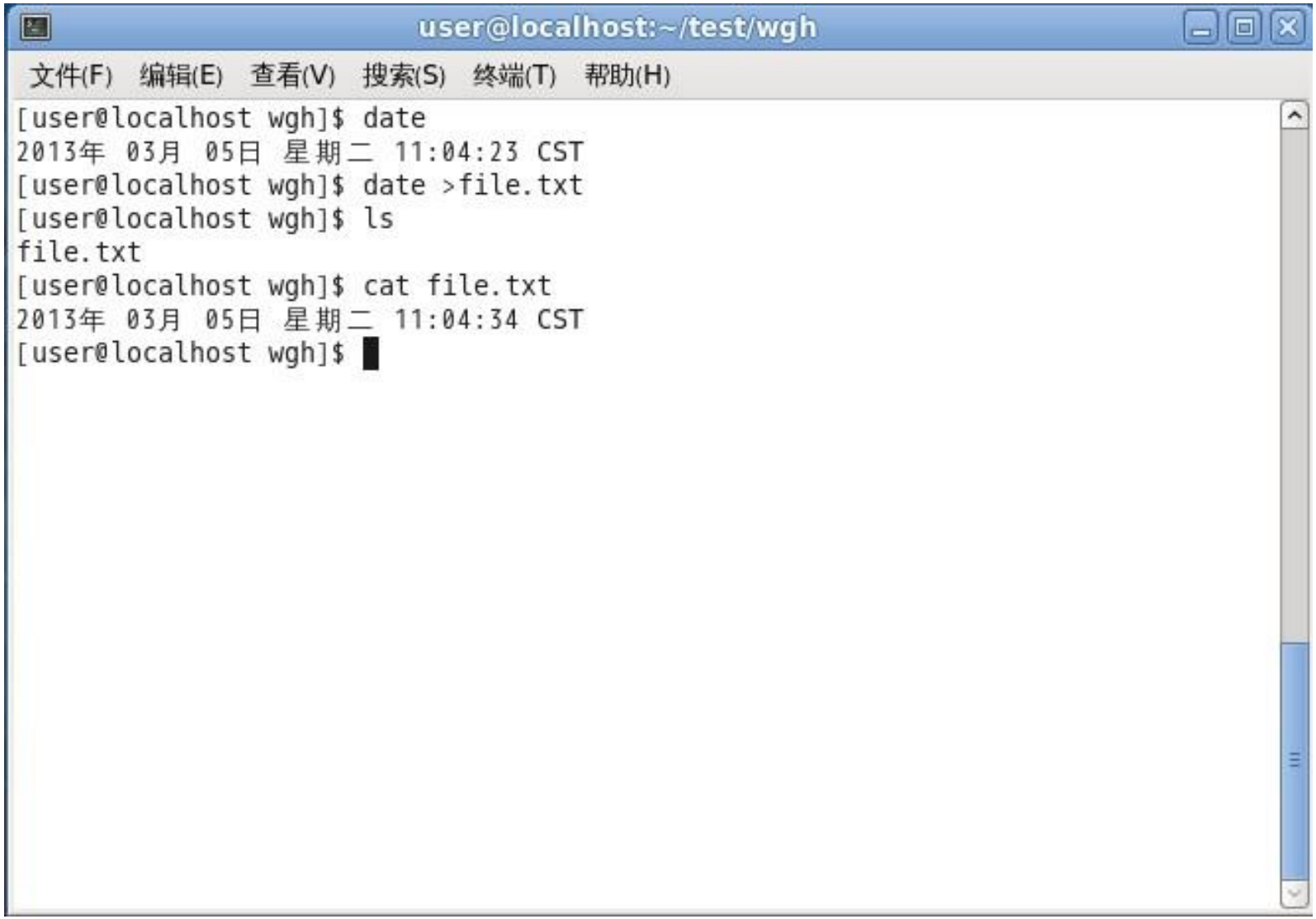
Operating System Concepts (Protection & Shell)



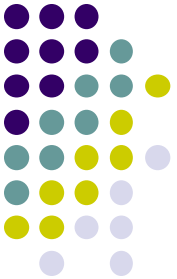
- date >file
- sort <file1 >file2
- cat file1 file2 file3 | sort >/dev/lp
- cat file1 file2 file3 | sort >/dev/lp &

Operating System Concepts (Protection & Shell)





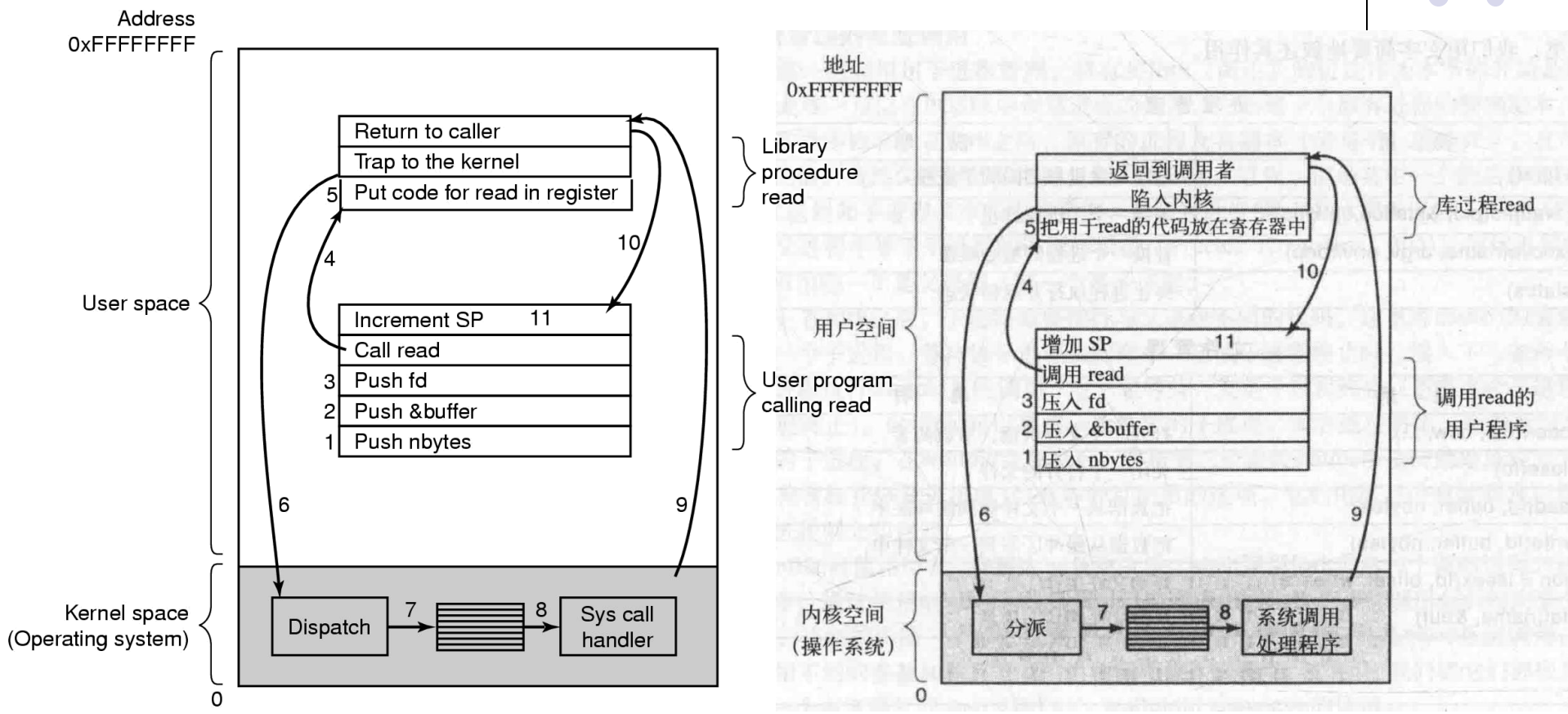
1. 6 System Calls



System calls provide the interface between a running program and the operating system

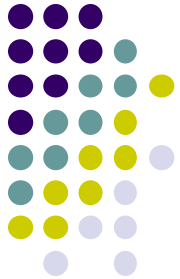
- Process management
- File management
- Directory or Device management
- Miscellaneous
  - Information maintenance
  - Communications

Steps in Making a System Call



There are 11 steps in making the system call
read (fd, buffer, nbytes)

Steps in Making a System Call (2)



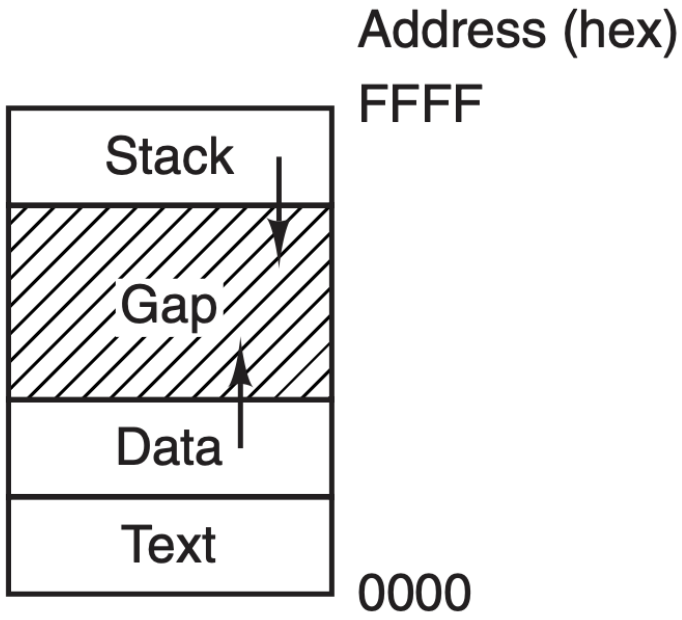


Figure 1-20. Processes have three segments: text, data, and stack.

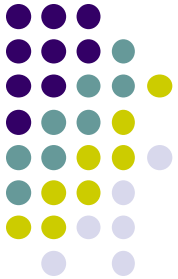


Some System Calls For Process Management

Process management

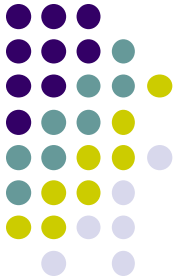
Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

System Calls for Process



- Fork creates a duplicate of the original process.
- UNIX examples
  - **fork** system call creates new process
  - **execve** system call used after a **fork** to replace the process' memory space with a new program
  - **waitpid** system call used to move the parent process off the ready queue until the termination of the child
- Exit system call is used when a process is finished executing

System Calls (Process)

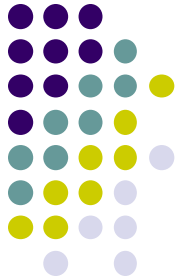


- A stripped down shell:

```
#define TRUE 1

while (TRUE) {                                  /* repeat forever */
        type_prompt( );                         /* display prompt on the screen */
        read_command(command, parameters);      /* read input from terminal */

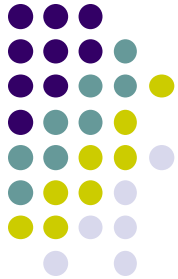
        if (fork( ) != 0) {                     /* fork off child process */
             /* Parent code. */
             waitpid(–1, &status, 0);           /* wait for child to exit */
        } else {
             /* Child code. */
             execve(command, parameters, 0);    /* execute command */
        }
}
```



Some System Calls For File Management

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

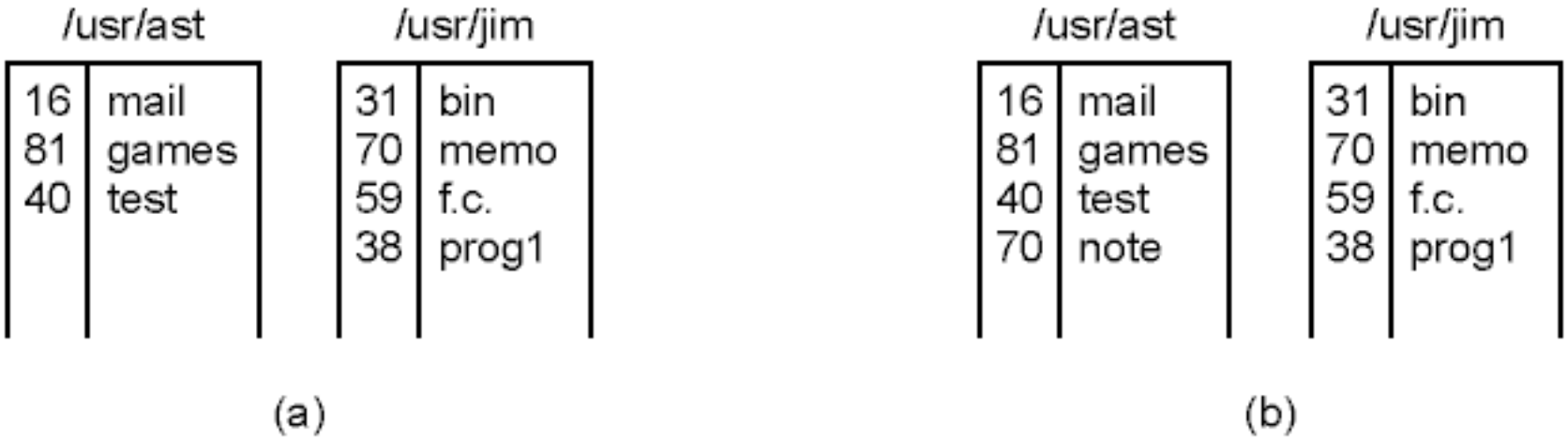


Some System Calls For Directory Management

Directory and file system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

System Calls (Linking)

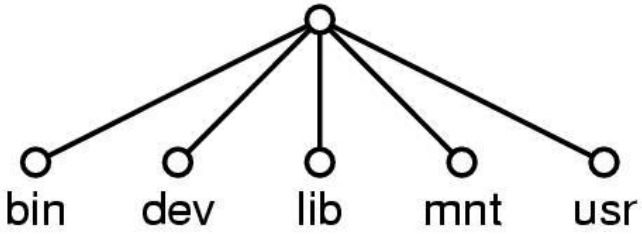


link("/usr/jim/memo", "/usr/ast/note");

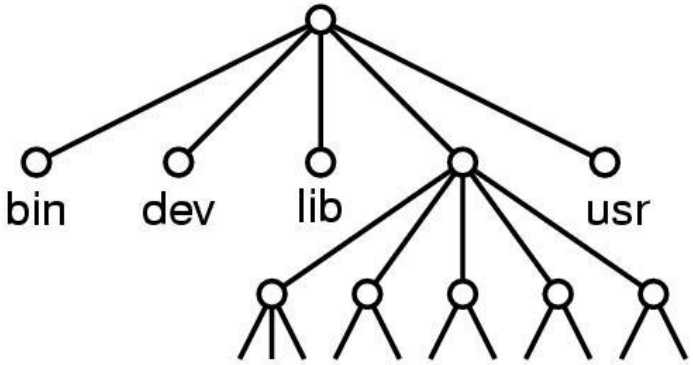
(a) Two directories before linking */usr/jim/memo* to ast's directory

(b) The same directories after linking

System Calls (Mounting)



(a)

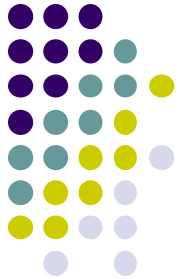


(b)

mount("/dev/fd0", "/mnt", 0)

(a) File system before the mount

(b) File system after the mount

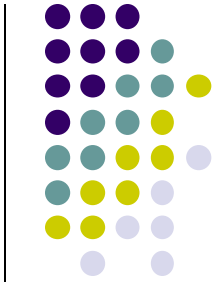


Some System Calls For Miscellaneous Tasks

Miscellaneous

Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970

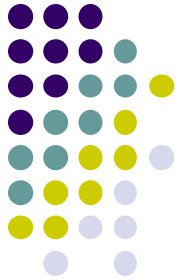
System Calls (Win32 API)



UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

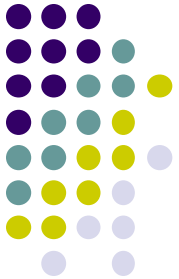
The Win32 API calls that roughly correspond to the UNIX calls of Fig. 1-18

1.7 Operating System Structure



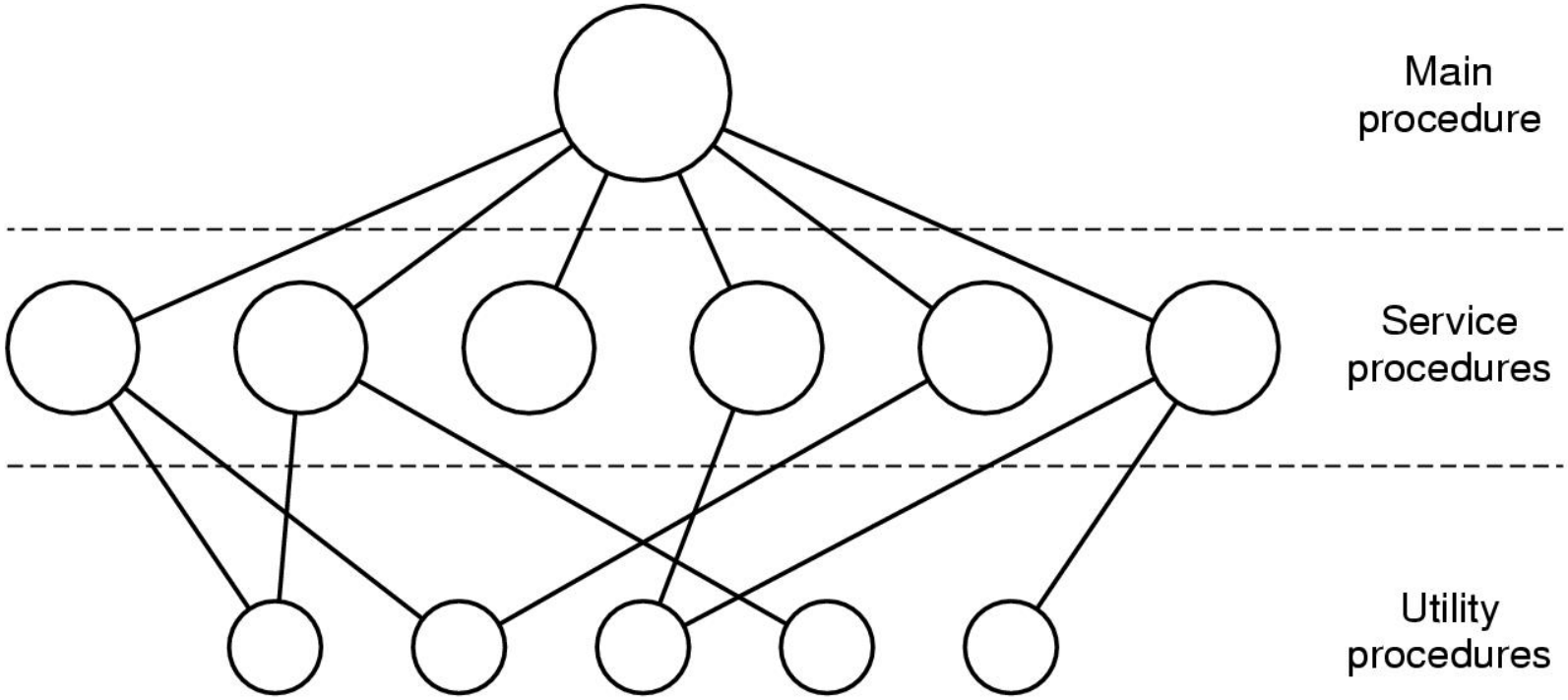
- Monolithic System - "the big mess"
  - All operating system operations are put into a single file. The operating system is a collection of procedures, each of which can call any of the others.
- Layered System
  - The operating system is organized as a hierarchy of layers of processes
- Client-server model or microkernel
  - micro-kernel and service processes
  - The operating system is collection of servers that provide specific services; e.g., file server, etc.
- Virtual Machine

Operating System Structure (Monolithic)



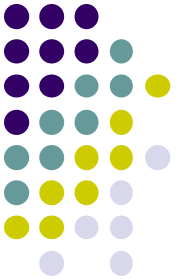
- Monolithic systems – basic structure:
  - A main program that invokes the requested service procedure.
  - A set of service procedures that carry out the system calls.
  - A set of utility procedures that help the service procedures.

Operating System Structure (Monolithic)



Simple structuring model for a monolithic system

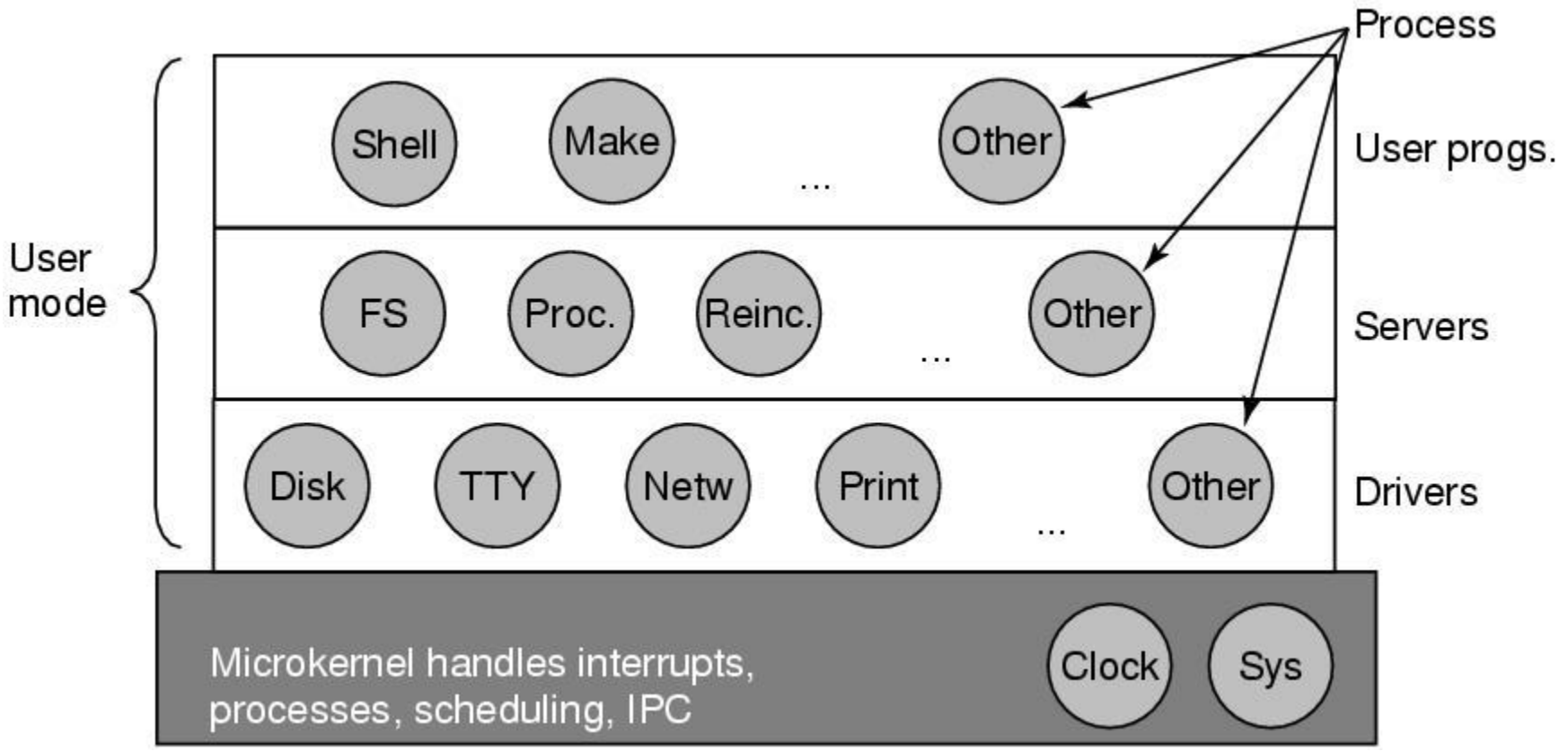
Operating System Structure (Layered)



Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

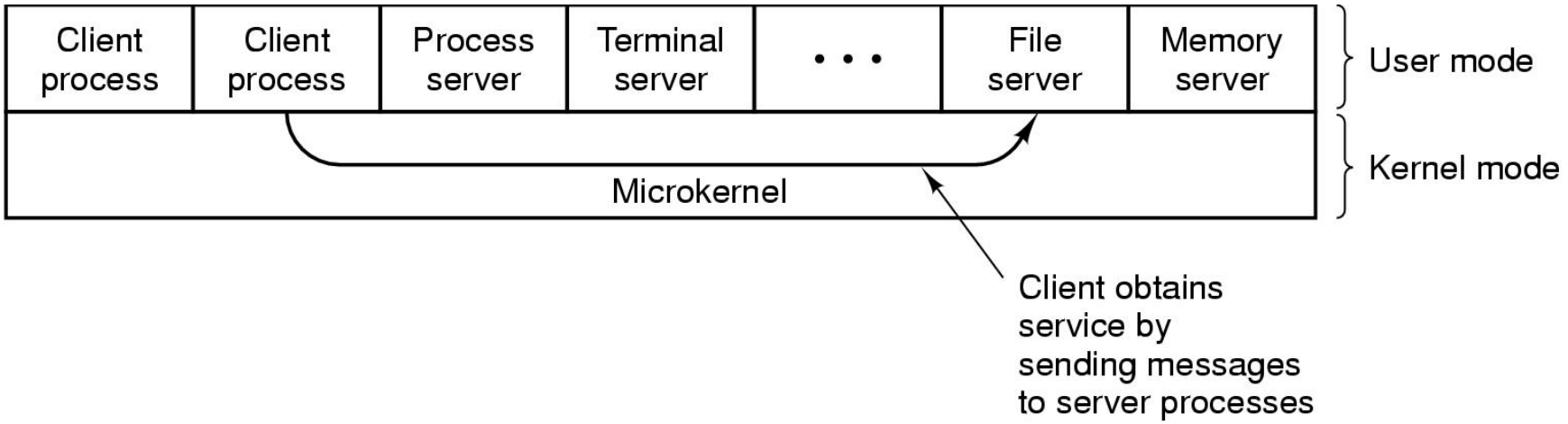
Structure of the THE operating system

Operating System Structure (Microkernels)



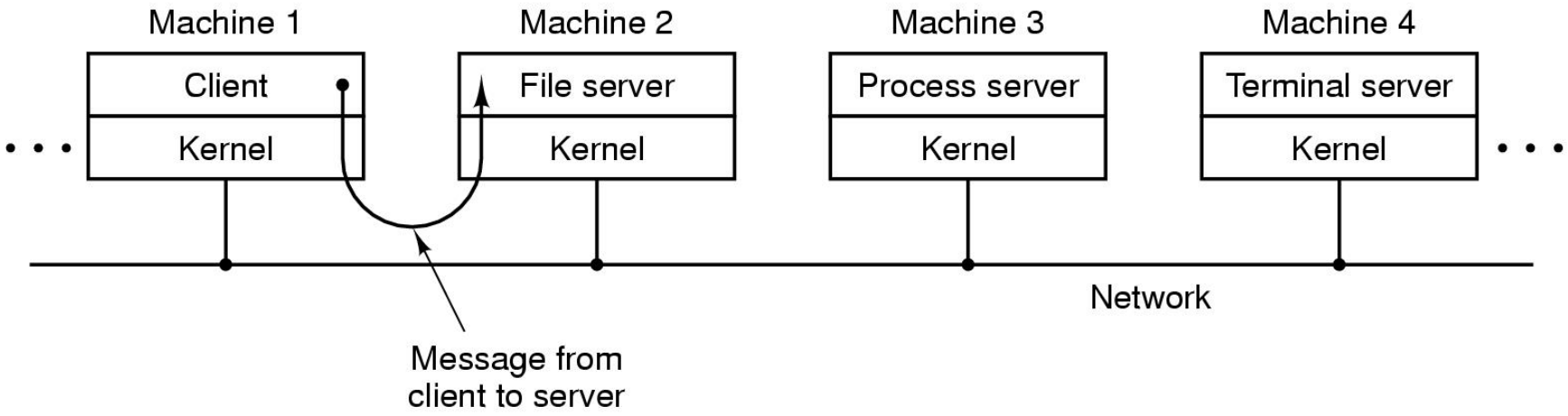
Structure of the MINIX 3 system

Operating System Structure (Client-server)



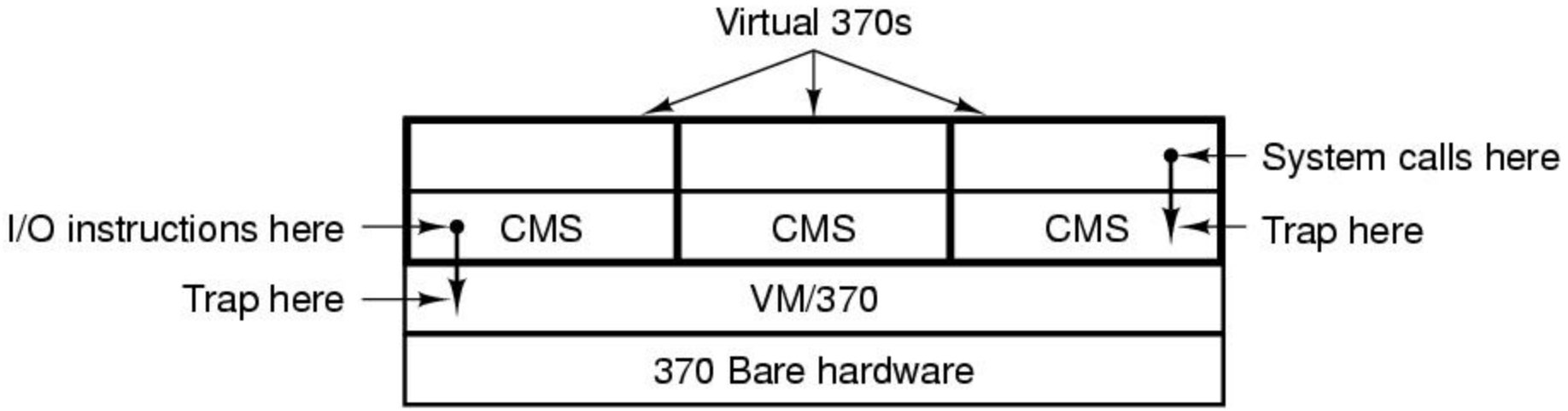
The client-server model

Operating System Structure (Client-Server)



The client-server model in a distributed system

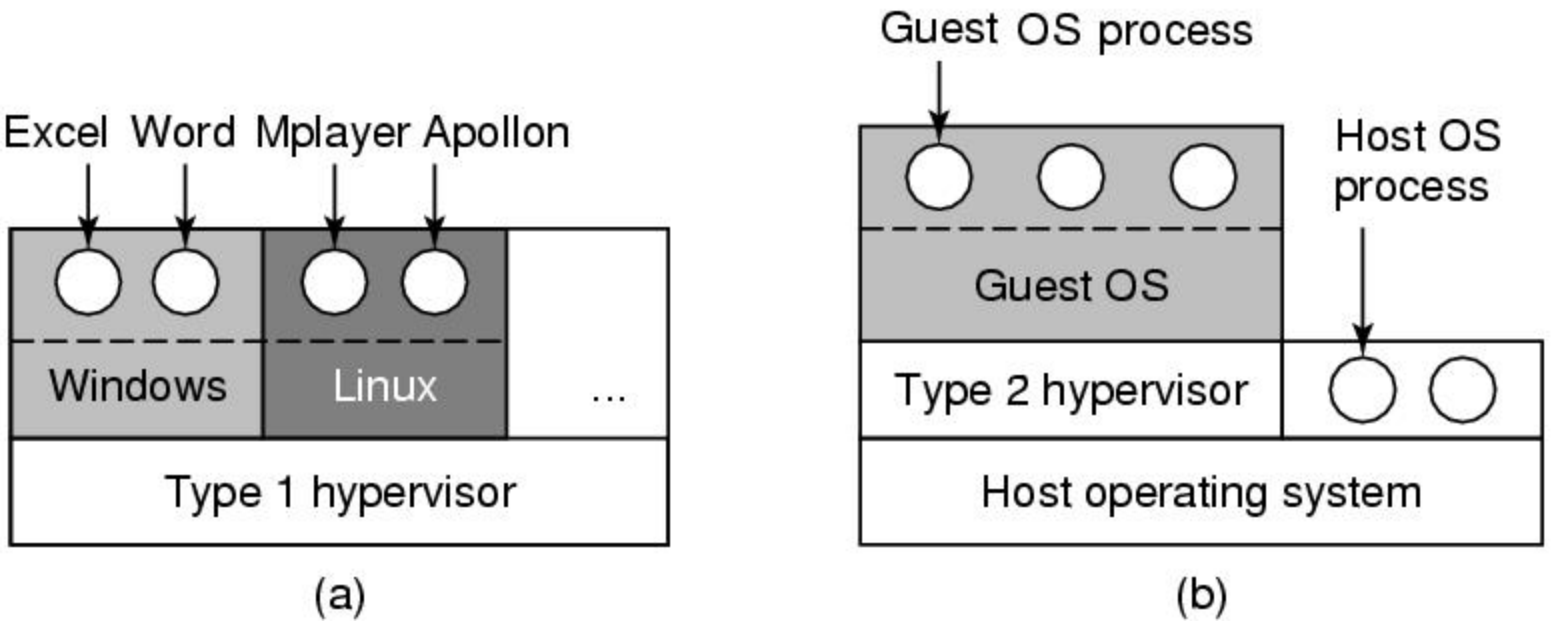
Operating System Structure (Virtual Machines)



The structure of VM/370 with CMS (Conversational Monitor System)

Tanenbaum, Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-6006639

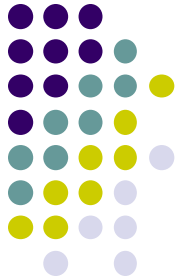
Operating System Structure (Virtual Machines)



(a) A type 1 hypervisor (b) A type 2 hypervisor

Tanenbaum, Modern Operating Systems 3 e, (c) 2008 Prentice-Hall, Inc. All rights reserved. 0-13-6006639

Summary



- Two viewpoints of OS: resource managers, and extended machines
- History of OS

  handwork → batch systems → multiprogramming systems → personal computer systems → mobile systems
- Computer hardware
- The operating system zoo
- Processes, Address Space, Files, and Protection
- System calls
- OS structures: Monolithic System, Layered System, C/S(microkernel), and Virtual Machine

Thank you!

